

Database security:

Database security means keeping your database safe from any harm, whether by mistake or on purpose. It's not just about protecting data. It also keeps the system and database structure safe.

To do this, security covers:

- **Hardware** (physical devices)
- **Software** (programs running the database)
- **People** (users who access the database)
- **Data** (stored information)

To keep everything secure, certain **rules and measures (controls)** are used. These help prevent unauthorized access, data loss, or damage.

In the past, businesses didn't focus much on database security, but now it's a big priority because data is very important.

Why Database Security is Important?

Security is an important concern in database management because the information stored in a database is a very valuable and, at times, quite sensitive commodity. **As a result, data in a database management system must be protected from abuse and illegal access and updates.**

- 1.Protects Sensitive Data** – Keeps important information safe from hackers, unauthorized users, and accidental loss.
- 2.Prevents Data Breaches** – Stops cybercriminals from stealing or leaking private data, which can harm a business.
- 3.Maintains Data Integrity** – Ensures that data remains accurate and unchanged by unauthorized users.
- 4.Ensures System Stability** – Prevents attacks or errors that could crash the database and disrupt operations.

5. Avoids Financial Loss – Data breaches can lead to heavy fines, legal issues, and loss of customer trust.

6. following laws– Many industries have strict rules (like GDPR, HIPAA) that require strong database security.

Examples of such rules:

- **GDPR** (in Europe) – Protects personal data.
- **HIPAA** (for healthcare) – Keeps medical records private.

7. Prevents Insider Threats – Limits access so only authorized employees can view or change important data.

8. Enables Business Continuity – With backups and security measures in place, businesses can recover quickly from attacks or failures.

A secure database protects valuable information, builds trust, and ensures smooth business operations.

Security and Integrity Violations:

In a Database Management System, security and integrity are essential to ensure that data remains **confidential, accurate, and consistent**. A violation occurs when data is accessed, modified, or destroyed in an unauthorized or incorrect manner.

1. Security Violations

Security violations refer to unauthorized access or misuse of database data. These may be:

- **Intentional (malicious):**
Examples include hacking, unauthorized data modification, data theft, or deliberate data deletion.
- **Unintentional:**
Occur due to user errors, such as incorrect query execution or misuse of privileges.

Security mechanisms such as authentication, authorization, access control, and encryption are used to prevent these violations.

2. Integrity Violations

Integrity violations occur when **data correctness and consistency are compromised**. These are usually accidental and may result from:

- System crashes during transaction processing, leaving the database in an inconsistent state.
- When multiple users access and change the same data at the same time, problems like lost updates or incorrect reads can happen if there's no control.
- Distributed database failures, where data is stored across multiple computers and network or node failures cause partial updates.

Integrity constraints, transaction management, concurrency control, and recovery mechanisms help maintain data integrity.

Access Control

Access Control is the mechanism that regulates **who can access** the database and **what operations they are allowed to perform on database** objects such as tables, views, and records. Its main objective is to protect data from unauthorized access, modification, or misuse, thereby ensuring database security and integrity.

Access Control Methods

1. Discretionary Access Control (DAC)

In DAC, the owner of a database object decides who can access it and what privileges they can have.

Features:

- Access rights are granted at the discretion of the object owner.
- Implemented using **GRANT** and **REVOKE** commands.

2. Mandatory Access Control (MAC)

In MAC, access decisions are made based on security levels assigned to users and data objects. Users cannot change access permissions.

Features:

- Used in high-security environments (e.g., military systems).
- Access depends on clearance levels such as *Confidential*, *Secret*, or *Top Secret*.

3. Role-Based Access Control (RBAC)

- In RBAC, permissions are assigned to roles, and users are assigned those roles.
- **Features:**
- Simplifies permission management.
- Common in enterprise systems.

• **Authorization**

Authorization in a Database Management System (DBMS) is the **process of granting or denying specific privileges to users or roles**, determining what actions an authenticated user is allowed to perform on database objects such as tables, views, and schemas. It is a key component of database security that controls access after a user has been authenticated.

How Authorization Helps in Security

- **Prevents Unauthorized Data Access**

Authorization ensures that users can access only the data for which they have been explicitly granted permission, protecting sensitive information.

- **Controls Database Operations**

Users are allowed to perform only permitted operations such as SELECT, INSERT, UPDATE, or DELETE, preventing accidental or malicious data modification.

- Implements Principle of Least Privilege**

Users are given only the minimum permissions they need. This reduces the chance of mistakes or data loss.

- Supports Role-Based Security**

Giving permissions to roles instead of individual users makes security easier to manage and keeps access rules consistent.

- Protects Data Integrity**

Restricting who can update or delete data helps keep it correct and consistent.

Authorization and Views:

Authorization

Authorization is the **process of granting or restricting access rights to users on database objects**. It ensures that only permitted users can perform specific operations such as SELECT, INSERT, UPDATE, and DELETE, thereby protecting data from unauthorized access.

- Authorization is implemented using SQL commands like **GRANT** and **REVOKE**.

Example:

```
GRANT SELECT, INSERT ON Student TO User1;
```

```
REVOKE INSERT ON Student FROM User1;
```

Views

A view in DBMS is a virtual table created using a SELECT query. It does not store data itself but shows data taken from one or more existing tables.

Example:

```
CREATE VIEW Student_View AS  
SELECT RollNo, Name, Department  
FROM Student;
```

How Views Help in Implementing Security

Views enhance database security by restricting user access to specific rows and columns of a table. Instead of granting access to the entire base table, users are given access only to the required view.

Security Benefits of Views:

- Hide sensitive columns (e.g., salary, password)
- Restrict rows using WHERE conditions
- Provide controlled and customized data access
- Prevent direct access to base tables

Example:

```
GRANT SELECT ON Student_View TO User2;
```

Encryption and Decryption:

In DBMS, encryption is the process of converting readable data (plain text) into an unreadable form (cipher text) using an algorithm and a key to protect it from unauthorized access.

Decryption is the reverse process, converting the cipher text back into plain text using a decryption algorithm and key, allowing authorized users to access the original data.

Types of Encryption

1. Symmetric Encryption (Secret-Key Cryptography)

- Symmetric encryption uses a single, secret key for both the encryption and decryption processes. This key must be shared securely between the communicating parties.

Key Features:

- **Key Usage:** The same key is used by the sender to encrypt and by the receiver to decrypt.
- **Speed:** It is very fast and efficient, making it ideal for encrypting large amounts of data (bulk data).
- **Security Challenge:** The primary challenge is the secure exchange of the single secret key between the parties. If an unauthorized party intercepts the key, all communication is compromised.

Common Algorithms:

- **Advanced Encryption Standard (AES):** The most widely used symmetric algorithm, approved by the U.S. government for top-secret information.
- **Data Encryption Standard (DES) / Triple DES (3DES):** Older standards, with 3DES still used in some financial sectors.

2. Asymmetric Encryption (Public-Key Cryptography)

- Asymmetric encryption uses a pair of mathematically related keys: a public key and a private key.

Key Features:

Two different keys are used:

- **Public Key:** Used for encryption. This key can be openly distributed to anyone.
- **Private Key:** Used for decryption. This key must be kept secret and private by the owner.

Relationship: Data encrypted with the public key can *only* be decrypted with the corresponding private key, and vice versa.

- **Speed:** It is significantly slower than symmetric encryption.
- **Security Advantage:** It solves the key distribution problem, as only the public key needs to be shared, while the secret private key never leaves the owner. It is also used for digital signatures and key exchange.

Common Algorithms:

- **RSA (Rivest–Shamir–Adleman):** One of the oldest and most common public-key cryptosystems, widely used for key exchange and digital signatures.
- **Elliptic Curve Cryptography (ECC):** Provides a similar level of security to RSA but with smaller key sizes, making it more efficient for mobile devices and performance-sensitive applications.

Example: Sending money via online banking or PayPal

- When you log in or send money online, the website (like your bank) gives you its **public key**.
- Your browser encrypts your password or transaction info using that public key.
- The bank receives the encrypted data and decrypts it using its private key.

Example: Email signature

- Alice writes an email.
- She locks the email with her private key (creates a digital signature).
- Bob receives it and unlocks it with Alice's public key.
- If it works, Bob knows the email really came from Alice and wasn't changed.

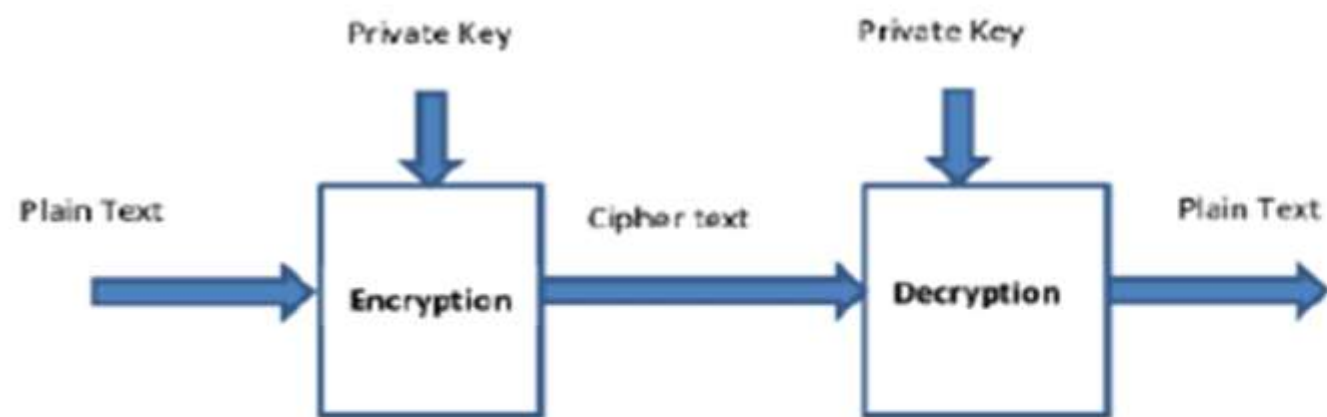


Fig : Private key Cryptography

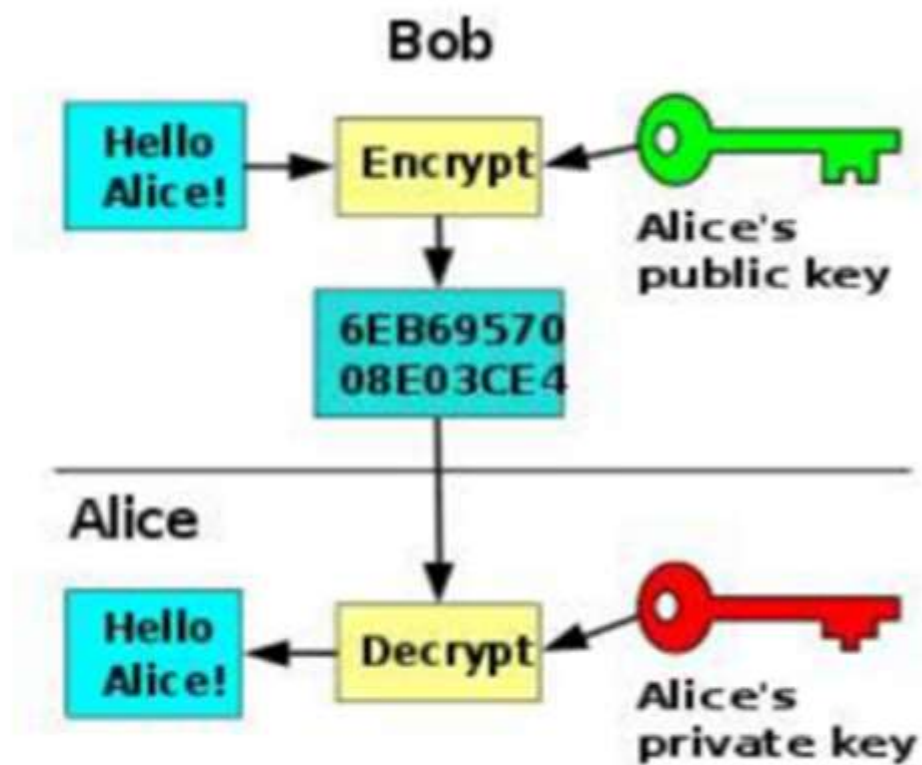


Fig: Public Key Cryptography

Need for Encryption

- Protect sensitive data like passwords, credit card info
- Prevent unauthorized access and hacking
- Ensure secure storage and transmission of data

Advantages

- High level of data security
- Maintains confidentiality and privacy
- Protects data even if database is compromised
- Prevents misuse of information